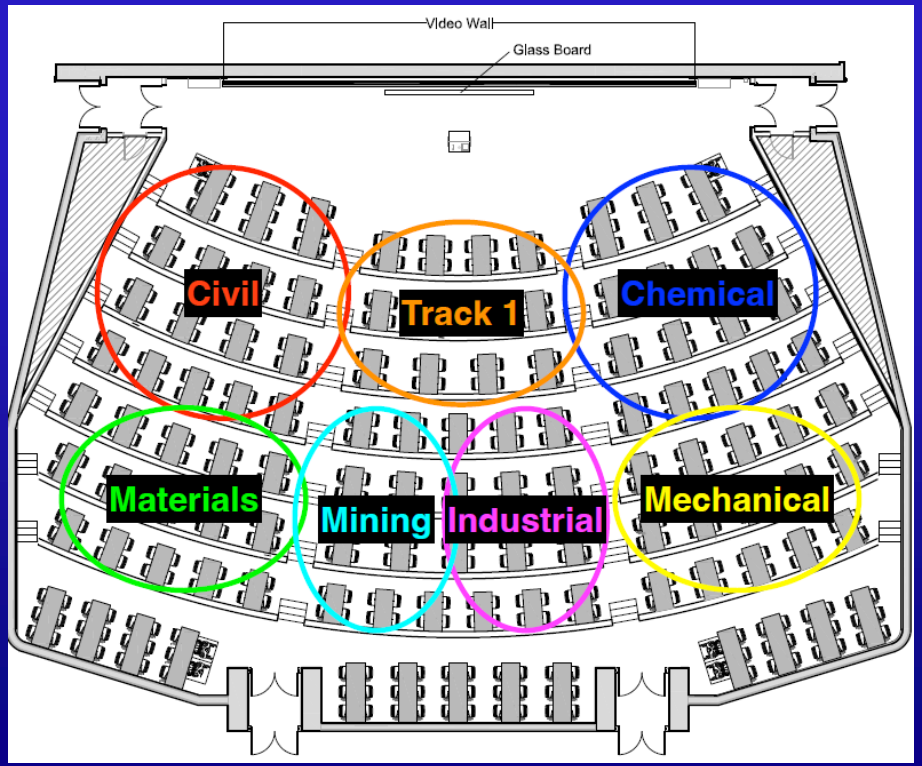
APS106

TERM TEST 2 REVIEW


Video Wall
Glass Board
Civil
Track 1
Chemical
Materials
Mining
Industrial
Mechanical

RULES OF THE GAME

- Must use button on the table to answer question verbally and explain your answer – you will be called on to answer by one of the instructors
- Must tell us which discipline you're in before you answer for points (we'll rely on the honour system)
- If you get the answer right, you pick the next category
- If you get the answer wrong, the next person whose hand is up can steal
- Everyone is here to learn and review for the midterm so be kind to everyone who answers!

PANEL

DIFFICULT DICTIONARIES	ALL THINGS INDEX	FOR A WHILE THERE...	FANTASTIC CONTAINERS AND WHERE TO FIND THEM
$250	$250	$250	$250
$500	$500	$500	$500
$750	$750	$750	$750
$1000	$1000	$1000	$1000

FINAL JEOPARDY

DIFFICULT DICTIONARIES · $250

What is the output?

```
d = {"Running":45, "Hiking":101, "Biking":29, "Walking":9000}

d['Running'] += 1
print(d['Running'])
```

← BACK TO PANEL

Question board

DIFFICULT DICTIONARIES · $250

What is the output?

```
d = {"Running":45, "Hiking":101, "Biking":29, "Walking":9000}

d['Running'] += 1
print(d['Running'])

# SOLUTION: 46
```

← BACK TO PANEL

Question board

DIFFICULT DICTIONARIES · $500

What is the output?

```
d = {"Running":45, "Hiking":101, "Biking":29, "Walking":9000}

del d['Walking']
print(d.pop('Running'))
print(list(d.values()))
```

← BACK TO PANEL

Question board

DIFFICULT DICTIONARIES · $500

What is the output?

```
d = {"Running":45, "Hiking":101, "Biking":29, "Walking":9000}

del d['Walking']
print(d.pop('Running'))
print(list(d.values()))

# SOLUTION:
# 45
# [101, 29]
```

← BACK TO PANEL

Question board

DIFFICULT DICTIONARIES · $750

What is the output?

```
dict_1 = {
    "class": {
        "student": {
            "name": "Mike",
            "marks": {
                "physics": {
                    "midterm": 67,
                    "exam": 81 }
            }
        }
    }
}

print("class" in dict_1)
print("name" in dict_1["class"])
print(dict_1["class"]["student"]["name"]["marks"]["physics"])
```

← BACK TO PANEL

Question board

DIFFICULT DICTIONARIES · $750

What is the output?

```
dict_1 = {
    "class": {
        "student": {
            "name": "Mike",
            "marks": {
                "physics": {
                    "midterm": 67,
                    "exam": 81 }
            }
        }
    }
}

print("class" in dict_1)
print("name" in dict_1["class"])
print(dict_1["class"]["student"]["name"]["marks"]["physics"])

# SOLUTION:
# True
# False
# ERROR
```

Question board

DIFFICULT DICTIONARIES · $1000

What is the output?

```
string = "so APS106 is so cool, isn't it?"
str_split = string.split(' ')
dictionary = {}

for s in (str_split):
    dictionary[s] = 1
print(dictionary)
```

← BACK TO PANEL

Question board

DIFFICULT DICTIONARIES · $1000

What is the output?

```
string = "so APS106 is so cool, isn't it?"
str_split = string.split(' ')
dictionary = {}

for s in (str_split):
    dictionary[s] = 1
print(dictionary)

# SOLUTION: it prints
# {'so': 1, 'APS106': 1, 'is': 1, 'cool,': 1, "isn't": 1, 'it?': 1}
```

← BACK TO PANEL

Question board

ALL THINGS INDEX · $250

What is the output?

```
string = "APS 106 is really cool!"
string_split = string.split(' ')

print(string_split[0] + string_split[1] + ' ' + string_split[4])

print(string_split)
```

← BACK TO PANEL

Question board

ALL THINGS INDEX · $250

What is the output?

```
string = "APS 106 is really cool!"
string_split = string.split(' ')

print(string_split[0] + string_split[1] + ' ' + string_split[4])

print(string_split)

# SOLUTION: APS106 cool!
```

← BACK TO PANEL

Question board

ALL THINGS INDEX · $500

What is the output?

```
colours = ['pink', 'blue', 'orange']
colours2 = colours
print(colours.extend(["Red", "yellow"]))
print(max(colours))
print(min(colours2))
```

← BACK TO PANEL

Question board

ALL THINGS INDEX · $500

What is the output?

```
colours = ['pink', 'blue', 'orange']
colours2 = colours
print(colours.extend(["Red", "yellow"]))
print(max(colours))
print(min(colours2))

# SOLUTION:
#None
#yellow
#Red
```

← BACK TO PANEL

Question board

ALL THINGS INDEX · $750

What is the output? Multiple Choice:

```
colours = ['red']
colours.append('pink')
colours.append(['orange'])
colours.extend(['blue','yellow'])
colours.extend('purple')
print(colours)

# Options:
A. ['red', 'pink', ['orange'], 'blue', 'yellow', 'p', 'u', 'r', 'p', 'l', 'e']
B. ['red', 'pink', ['orange'], 'blue', 'yellow', 'purple']
C. ['red', 'pink', 'orange', ['blue', 'yellow'], 'purple']
D. ['red', 'pink', ['orange'], ['blue', 'yellow'], 'purple']
E. ['red', 'pink', 'orange', 'blue', 'yellow', 'purple'']
```

← BACK TO PANEL

Question board

ALL THINGS INDEX · $750

What is the output? Multiple Choice:

```
colours = ['red']
colours.append('pink')
colours.append(['orange'])
colours.extend(['blue','yellow'])
colours.extend('purple')
print(colours)

# Options:
A. ['red', 'pink', ['orange'], 'blue', 'yellow', 'p', 'u', 'r', 'p', 'l', 'e']
```

← BACK TO PANEL

Question board

ALL THINGS INDEX · $1000

What is the output?

```
super_list = [
    ["Is", "APS", "106", "super lame", "?"],
    ["APS", "112", "is", "the most fun I've ever had", "yes definitely!"]
]

print(f"How do I feel about APS 106? {super_list[0][3].split(' ')[0] + super_list[1][-1][-1]}")
```

← BACK TO PANEL

Question board

ALL THINGS INDEX · $1000

What is the output?

```
super_list = [
    ["Is", "APS", "106", "super lame", "?"],
    ["APS", "112", "is", "the most fun I've ever had", "yes definitely!"]
]

print(f"How do I feel about APS 106? {super_list[0][3].split(' ')[0] + super_list[1][-1][-1]}")

# SOLUTION: How do I feel about APS 106? super!
```

← BACK TO PANEL

Question board

FOR A WHILE THERE ... · $250

What indices are ‘B’ and ‘C’ at?

```
ls = [
    ['A', 'A', 'A', 'A'],
    ['A', 'A', 'A', 'E', 'B'],
    ['A', 'A', 'A'],
    ['A', 'C', 'A', 'D']
]
```

← BACK TO PANEL

Question board

FOR A WHILE THERE ... · $250

What indices are ‘B’ and ‘C’ at?

```
ls = [
    ['A', 'A', 'A', 'A'],
    ['A', 'A', 'A', 'E', 'B'],
    ['A', 'A', 'A'],
    ['A', 'C', 'A', 'D']
]

print(ls[1][4]) #B
print(ls[3][1]) #C
```

← BACK TO PANEL

Question board

FOR A WHILE THERE ... · $500

What is the output?

```
test_string = "A3DE4"
for i in test_string:
    if i.isdigit():
        print(int(i)**2)
```

← BACK TO PANEL

Question board

FOR A WHILE THERE ... · $500

What is the output?

```
test_string = "A3DE4"
for i in test_string:
    if i.isdigit():
        print(int(i)**2)

# SOLUTION:
# 9
# 16
```

← BACK TO PANEL

Question board

FOR A WHILE THERE ... · $750

What is the output?

```
ls = [
    [1, 1, 1, 1, 2],
    [1, 1, 1, 1, 2],
    [1, 16, 1, 1, 2],
    [2, 1, 1, 1, 4],
]
counts = []
ncol = len(ls[0])
nrow = len(ls)
```

```
for col in range(ncol):
    c = 0
    for row in range(nrow):
        c += ls[row][col]
    counts.append(c)
print(counts)
```

← BACK TO PANEL

Question board

FOR A WHILE THERE ... · $750

What is the output?

```
ls = [
    [1, 1, 1, 1, 2],
    [1, 1, 1, 1, 2],
    [1, 16, 1, 1, 2],
    [2, 1, 1, 1, 4],
]
counts = []
ncol = len(ls[0])
nrow = len(ls)
```

```
for col in range(ncol):
    c = 0
    for row in range(nrow):
        c += ls[row][col]
    counts.append(c)
print(counts)

# SOLUTION: [5, 19, 4, 4, 10]
```

← BACK TO PANEL

Question board

FOR A WHILE THERE ... · $1000

What is the output?

```
my_food = ['apples', 'potato', 'pears', 'pomegranate']

dictionary = {}
for food in my_food:
    count = 0
    for letter in food:
        if letter >= 'a' and letter < 'f':
            count += 1
    dictionary[food] = count

print(dictionary)
```

← BACK TO PANEL

Question board

FOR A WHILE THERE ... · $1000

What is the output?

```
my_food = ['apples', 'potato', 'pears', 'pomegranate']

dictionary = {}
for food in my_food:
    count = 0
    for letter in food:
        if letter >= 'a' and letter < 'f':
            count += 1
    dictionary[food] = count

print(dictionary)

# SOLUTION:
# {'apples': 2, 'potato': 1, 'pears': 2, 'pomegranate': 4}
```

← BACK TO PANEL

Question board

FANTASTIC CONTAINERS AND WHERE TO FIND THEM · $250

Which of the following are mutable?

a. Strings
b. Tuples
c. Lists
d. Dictionaries
e. Sets

← BACK TO PANEL

Question board

FANTASTIC CONTAINERS AND WHERE TO FIND THEM · $250

Which of the following are mutable?

c. Lists
d. Dictionaries*
e. Sets

← BACK TO PANEL

Question board

FANTASTIC CONTAINERS AND WHERE TO FIND THEM · $500

What is the output?

```
tupl = (5, 2, 72)
x, y, z = tupl

print(x ** y + z + len(tupl))
```

← BACK TO PANEL

Question board

FANTASTIC CONTAINERS AND WHERE TO FIND THEM · $500

What is the output?

```
tupl = (5, 2, 72)
x, y, z = tupl

print(x ** y + z + len(tupl))

# SOLUTION: 100
```

← BACK TO PANEL

Question board

FANTASTIC CONTAINERS AND WHERE TO FIND THEM · $750

What is the output? Multiple Choice

```
def power_up_info():
    power_ups = ('Mushroom', 'Fire Flower', 'Super Star', '1-Up', 'Mega Mushroom')
    return power_ups, len(power_ups)

new_power_ups, count = power_up_info()
a, b, c, d, e = new_power_ups
cde = [c, d, e]

print(a, count, cde[::-2])

#A) ValueError: too many values to unpack
#B) Mushroom 5 ('Mega Mushroom', 'Super Star')
#C) TypeError: 'tuple' object does not support item assignment
#D) Mushroom 5 ['Mega Mushroom', 'Super Star']
```

Question board

FANTASTIC CONTAINERS AND WHERE TO FIND THEM · $750

What is the output? Multiple Choice

```
def power_up_info():
    power_ups = ('Mushroom', 'Fire Flower', 'Super Star', '1-Up', 'Mega Mushroom')
    return power_ups, len(power_ups)

new_power_ups, count = power_up_info()
a, b, c, d, e = new_power_ups
cde = [c, d, e]

print(a, count, cde[::-2])

#A) ValueError: too many values to unpack
#B) Mushroom 5 ('Mega Mushroom', 'Super Star')
#C) TypeError: 'tuple' object does not support item assignment
D) Mushroom 5 ['Mega Mushroom', 'Super Star']
```

← BACK TO PANEL

Question board

FANTASTIC CONTAINERS AND WHERE TO FIND THEM · $1000

What is the output?

```
warp_zones = (
    ('Underground', ['Level 1', 'Level 2'], 100),
    ('Sky', ['Level 2', 'Level 3', 'Final Level'], 200),
    ('Castle', ['Level 3', 'Level 4'], 300),
    ('Underwater', ['Level 3', 'Level 4', 'Final Level'], 400),
)

x = warp_zones[-1][1][-1]
for warp in warp_zones:
    if x in warp[1]:
        print(warp[0])
```

← BACK TO PANEL

Question board

FANTASTIC CONTAINERS AND WHERE TO FIND THEM · $1000

What is the output?

```
warp_zones = (
    ('Underground', ['Level 1', 'Level 2'], 100),
    ('Sky', ['Level 2', 'Level 3', 'Final Level'], 200),
    ('Castle', ['Level 3', 'Level 4'], 300),
    ('Underwater', ['Level 3', 'Level 4', 'Final Level'], 400),
)

x = warp_zones[-1][1][-1]
for warp in warp_zones:
    if x in warp[1]:
        print(warp[0])

# SOLUTION:
# Sky
# Underwater
```

Question board

FINAL JEOPARDY

FINAL JEOPARDY

Write a function `filter_and_organize_books` that:

- Takes a **list of book tuples** (`book_list`) and a float (`min_price`).
  - Each tuple contains: = `(title (str), author (str), price (float), genre (str))` .
- **Keeps only books** with `price >= min_price` .
- Organizes them into a dictionary `{genre: [book titles]}` , where:
  - **Keys** are genres.
  - **Values** are lists of book titles.
  - A **genre is added only if at least one book meets the price condition.**
- **Returns** the dictionary. The function **must not print anything**.

Example input: `books = [("The Hobbit", "J.R.R. Tolkien", 12.99, "Fantasy"), ("A Brief History of Time", "Stephen Hawking", 15.50, "Science"), ("The Great Gatsby", "F. Scott Fitzgerald", 10.99, "Classic"), ("1984", "George Orwell", 9.99, "Dystopian"), ("The Catcher in the Rye", "J.D. Salinger", 8.99, "Classic"), ("The Theory of Everything", "Stephen Hawking", 14.75, "Science"), ("Brave New World", "Aldous Huxley", 11.50, "Dystopian"),]`

Expected output: `{ 'Fantasy':['The Hobbit'], 'Science': ['A Brief History of Time', 'The Theory of Everything'], 'Classic': ['The Great Gatsby'], 'Dystopian': ['Brave New World'] }`

← BACK TO PANEL

FINAL JEOPARDY - ANSWER

```python
def filter_and_organize_books(book_list, min_price):
    '''
    List(Tuple(str, str, float, str)) -> Dictionary(str:list(str))
    '''
    dictionary = {}

    # iterate through all book tuples in our list
    for book_tuple in book_list:
        # note that our tuple is (title, author, price, genre)
        # you can decompose this or keep it as a tuple and index say book_tuple[0] as you go to get individual elements
        # I'll break it down and assign to variables (title, author, price, genre) for ease of reading
        (title, author, price, genre) = book_tuple

        # check, is the price (index 2) greater than the minimum?
        if price > min_price:
            # if so, include
            # check if genre is already in dictionary
            if genre in dictionary.keys():
                # if so, just append title to our list
                dictionary[genre].append(title)
            else:
                # if not, create a new list with only our new title
                dictionary[genre] = [title]

    # Return!
    return dictionary
```